

SEMANA 06

Comunicação com o Mundo Exterior

Fetch API, JSON e Programação Assíncrona



Escopo da Semana



Bloco A: Teoria e Prática Guiada

O que é API, Síncrono vs. Assíncrono, Sintaxe Fetch, Info CEP



Bloco B: Mão na Massa!

Validador Avançado de CEP, Portal de Notícias IBGE, CRUD de Contatos



Metodologia de Entrega

Regras para submissão dos 9 arquivos finais

O que é uma API?

A Interface de Programação de Aplicações



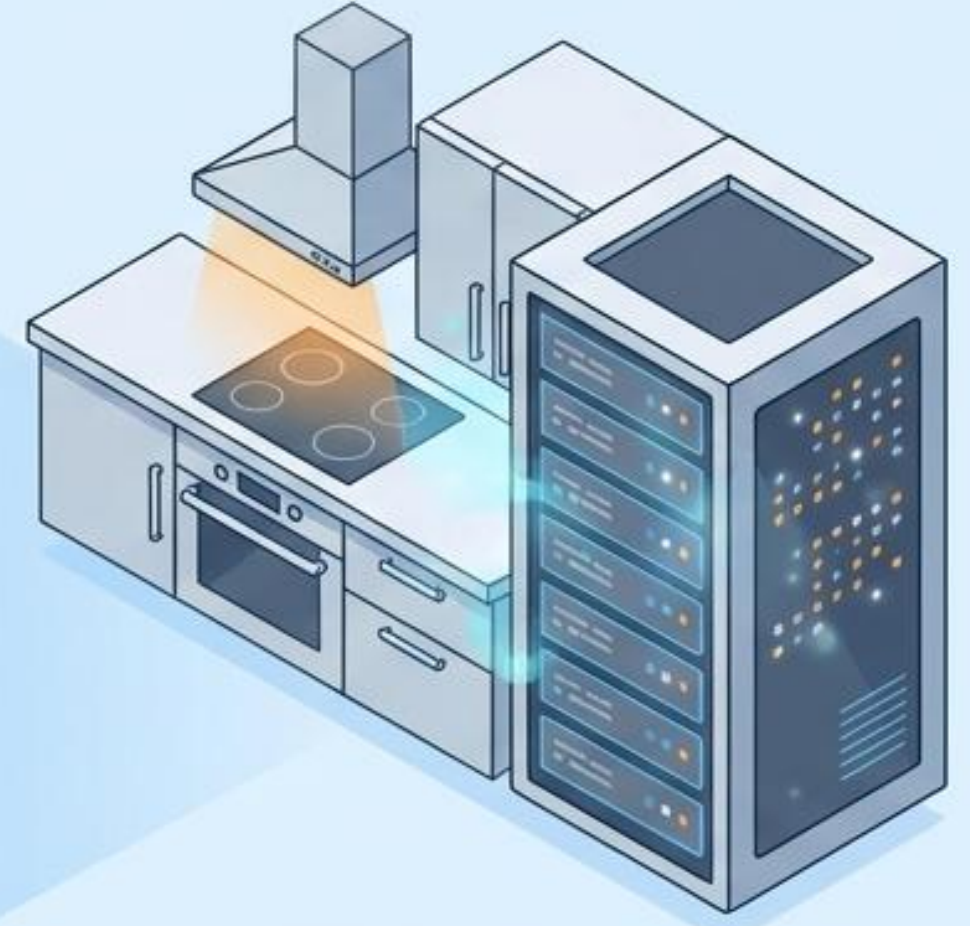
Você (App):

Quer os dados, mas não tem acesso direto aos bastidores.



O Garçom (A API):

O intermediário que anota o seu pedido, leva à cozinha e traz a resposta.



Cozinha (Servidor):

Onde os dados são processados e armazenados em segurança.

E o que vem na bandeja? O JSON. O formato de texto leve e universal que todas as linguagens entendem.

Síncrono vs. Assíncrono

Síncrono (A Fila do Banco)



- Você entra na fila e espera.
- Não pode fazer mais nada até ser atendido.
- **No Código:** Uma tarefa bloqueia a execução da próxima. A página congela.

Assíncrono (O Pager do Restaurante)



- Faz o pedido, recebe um pager e continua a conversar.
- O pager avisa apenas quando o pedido está pronto.
- **No Código:** A requisição inicia, o resto da página flui. Somos avisados quando a resposta chega.

A Ferramenta: Fetch API

1. O Pedido: Fazemos o pedido ao 'Garçom' passando a URL desejada.

2. A Tradução: Pegamos a resposta bruta e a transformamos em formato JSON.



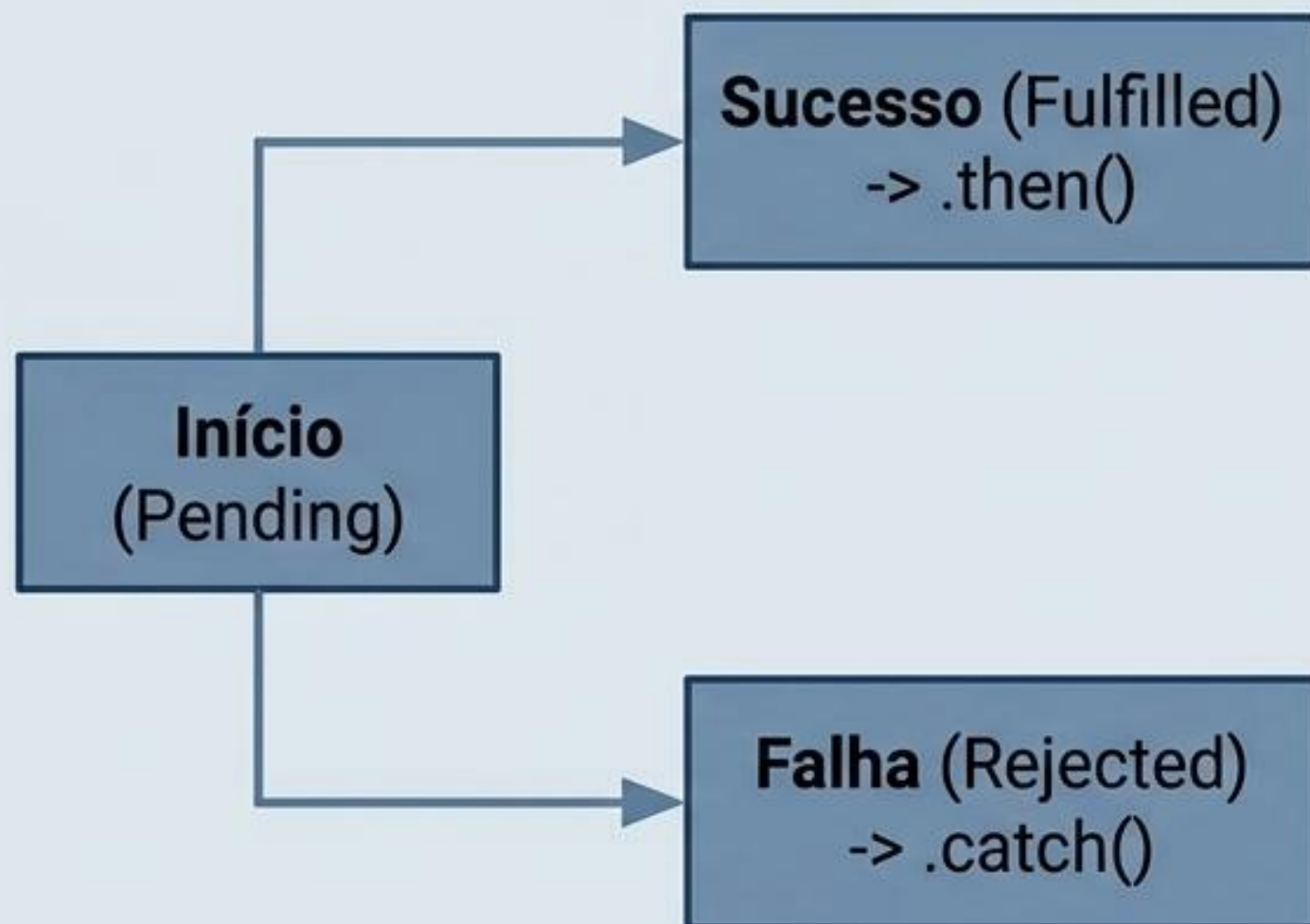
```
fetch('https://api.exemplo.com/dados')  
  .then(resposta => resposta.json())  
  .then(dados => { console.log(dados); })  
  .catch(erro => { console.error(erro); })
```

3. O Consumo: Usamos os dados prontos no nosso Front-End (na interface).

4. O Plano B: O que fazer se a internet cair ou o servidor falhar.

Trabalhando com Promises

O fetch não entrega os dados na hora; ele retorna uma *Promessa* de que a resposta chegará no futuro.



Abordagem 1 (.then)	Abordagem 2 (async / await)
Baseado no encadeamento de funções sucessivas.	O formato moderno. Lido de cima para baixo como código síncrono, exigindo uma função marcada com 'async'.

Prática Guiada: Info CEP (Bloco A)

Objetivo

Construir em conjunto uma aplicação que consome a API da ViaCEP para preencher automaticamente um endereço.

Toolkit

- `document.querySelector()` para capturar o input.
- `fetch(url)` com a URL dinâmica:
`https://viacep.com.br/ws/${cep}/json/`
- Manipulação de DOM para injetar Bairro, Cidade e Estado.

Visão Visual



Atividade Individual

Atividade 1: Validador Avançado de CEP

Do Zero ao Produto Final

Agora é com você. Crie uma ferramenta de busca robusta. Não basta apenas buscar os dados; o seu sistema deve ser à prova de erros de digitação, lidar com quedas de rede e fornecer feedback visual imediato ao usuário.

✓

Resultado Encontrado

Logradouro: Rua das Flores, 123 - Centro

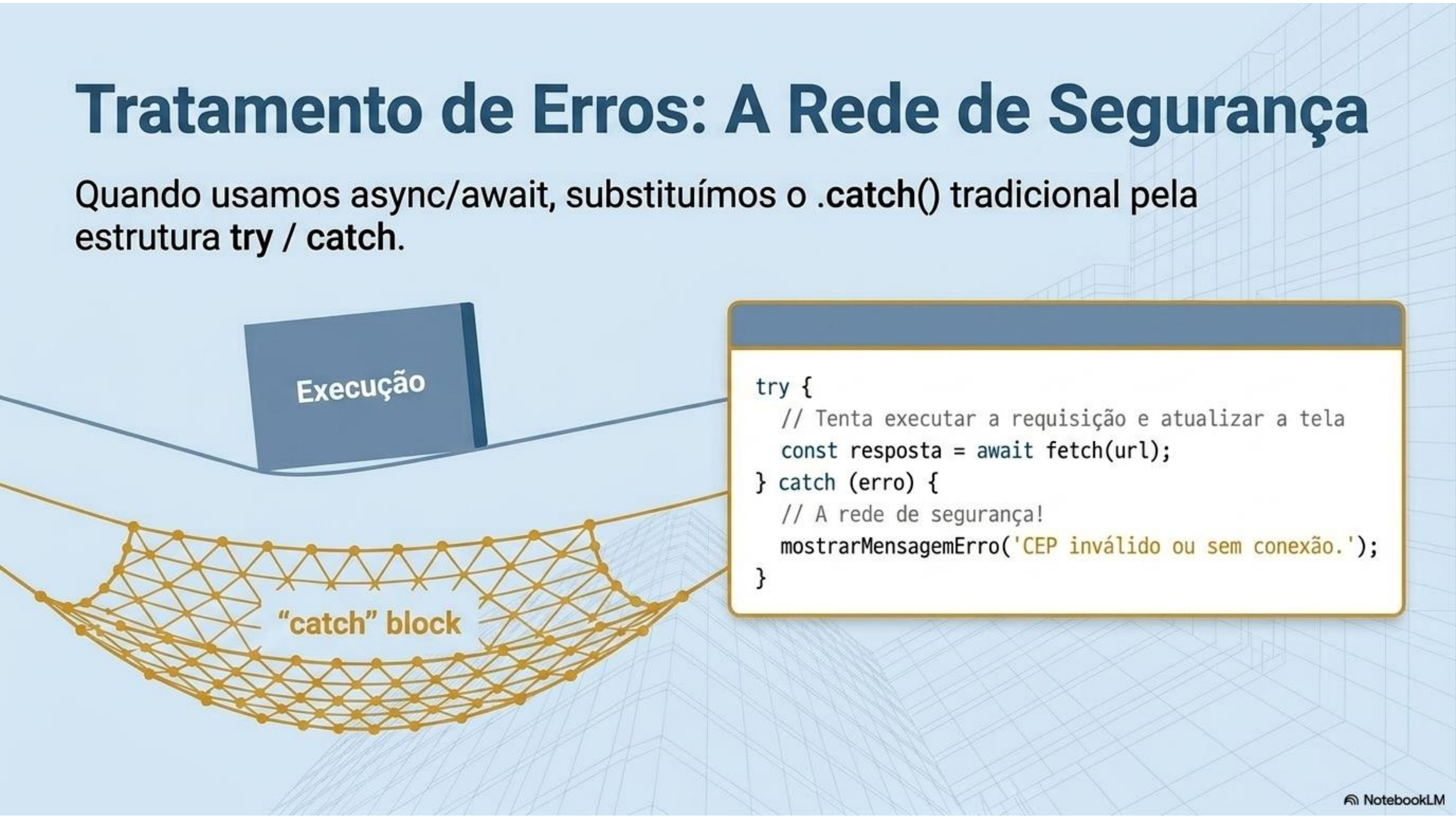
Bairro: Consolação

Cidade: São Paulo - SP

Limpar e Nova Busca

Tratamento de Erros: A Rede de Segurança

Quando usamos async/await, substituímos o `.catch()` tradicional pela estrutura `try / catch`.



The diagram features a blue rectangular block labeled 'Execução' (Execution) at the top left. A line extends from the bottom of this block towards a large, orange, diamond-shaped safety net below. The net is composed of a grid of interconnected lines and dots, with the text '“catch” block' written in the center. The background is a light blue with a faint grid pattern.

Execução

```
try {  
  // Tenta executar a requisição e atualizar a tela  
  const resposta = await fetch(url);  
} catch (erro) {  
  // A rede de segurança!  
  mostrarMensagemErro('CEP inválido ou sem conexão.');
```


Requisitos e Entregas: Atividade 1

Estrutura (HTML)	Interface (CSS)	Lógica (JS)
✓ Campo de input para exatos 8 dígitos. ✓ Botão de ação (Buscar). ✓ Área de resultado dedicada (vazia inicialmente).	✓ Layout limpo e centralizado. ✓ Feedback visual: O botão deve mudar de cor ou exibir texto "Carregando..." durante o fetch.	✓ Validar se o input está vazio antes do fetch. ✓ Envolver o fetch em try/catch. ✓ Tratar o caso específico em que a ViaCEP retorna { "erro": true }.

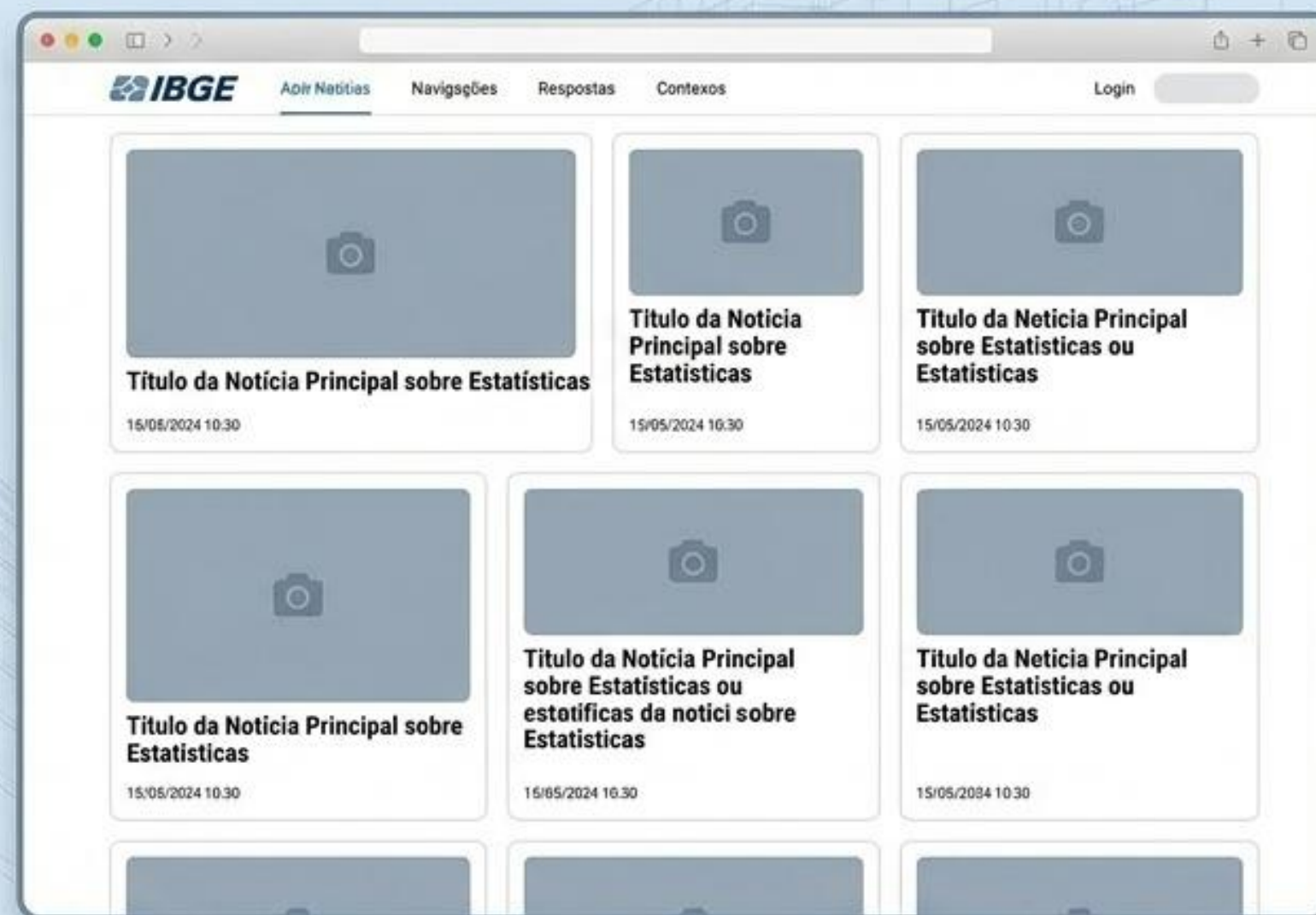
Atividade Individual

Atividade 2: Portal de Notícias do IBGE

Notícias do IBGE

Consumindo dados governamentais em tempo real.

Use a API pública de Notícias do IBGE (servicodados.ibge.gov.br). Ao contrário do CEP, que traz um único endereço, esta API retorna uma lista com dezenas de notícias. O desafio é renderizar todas elas na tela automaticamente.



Renderizando Arrays: A Máquina de Repetição

Quando a resposta é um Array, você não escreve o HTML 20 vezes. Você cria um modelo (Template String) e usa um **loop** para gerar os blocos de HTML dinamicamente com base nos dados.



Ferramentas necessárias:

```
>_ array.map() ou
</> for (let noticia of noticias)
```

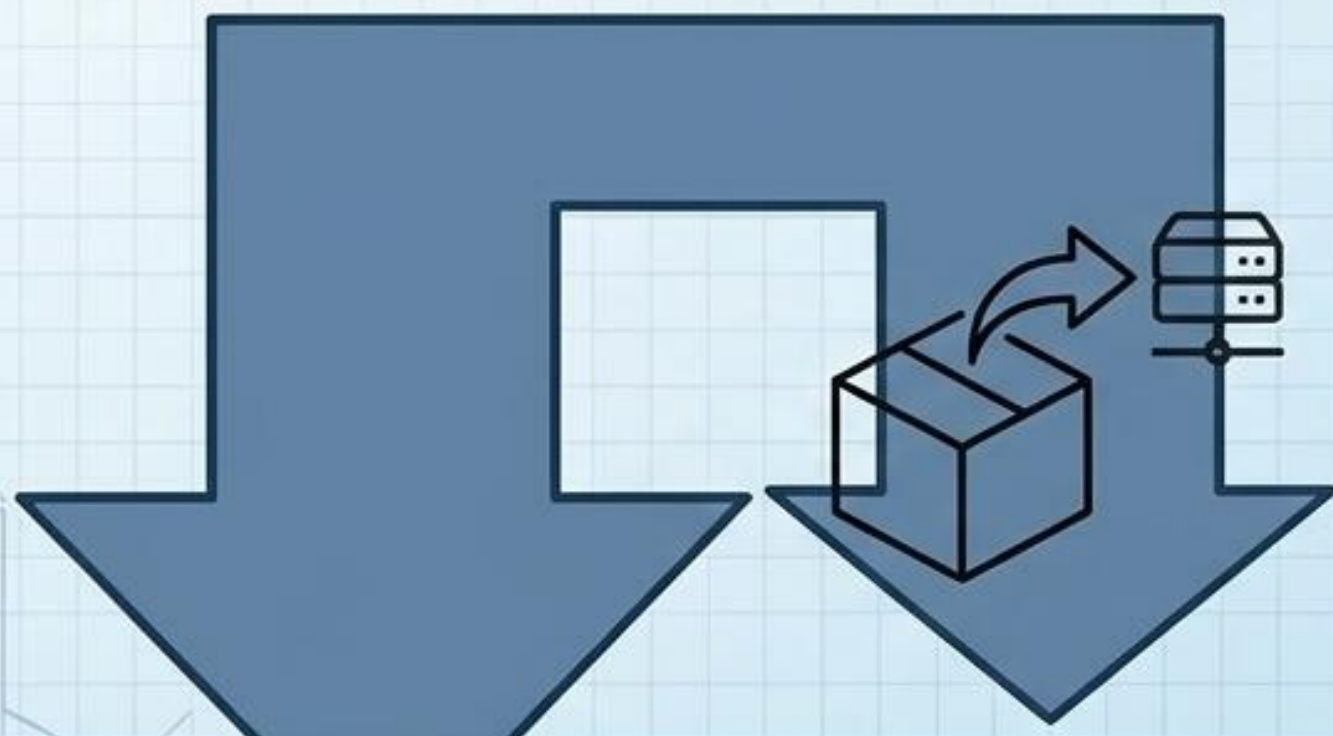

Requisitos e Entregas: Atividade 2

Conexão (Fetch)	Processamento (Arrays)	Renderização (DOM)
✓ Realizar o GET na URL correta do IBGE. ✓ Extrair corretamente o array de notícias de dentro do objeto de resposta JSON.	✓ Iterar sobre o array de dados. ✓ Extrair apenas as propriedades necessárias: Título e Data de publicação.	✓ Criar a Template String de cada notícia. ✓ Usar innerHTML para injetar os cartões HTML na página. ✓ Aplicar CSS Grid ou Flexbox para organizar o visual em colunas.

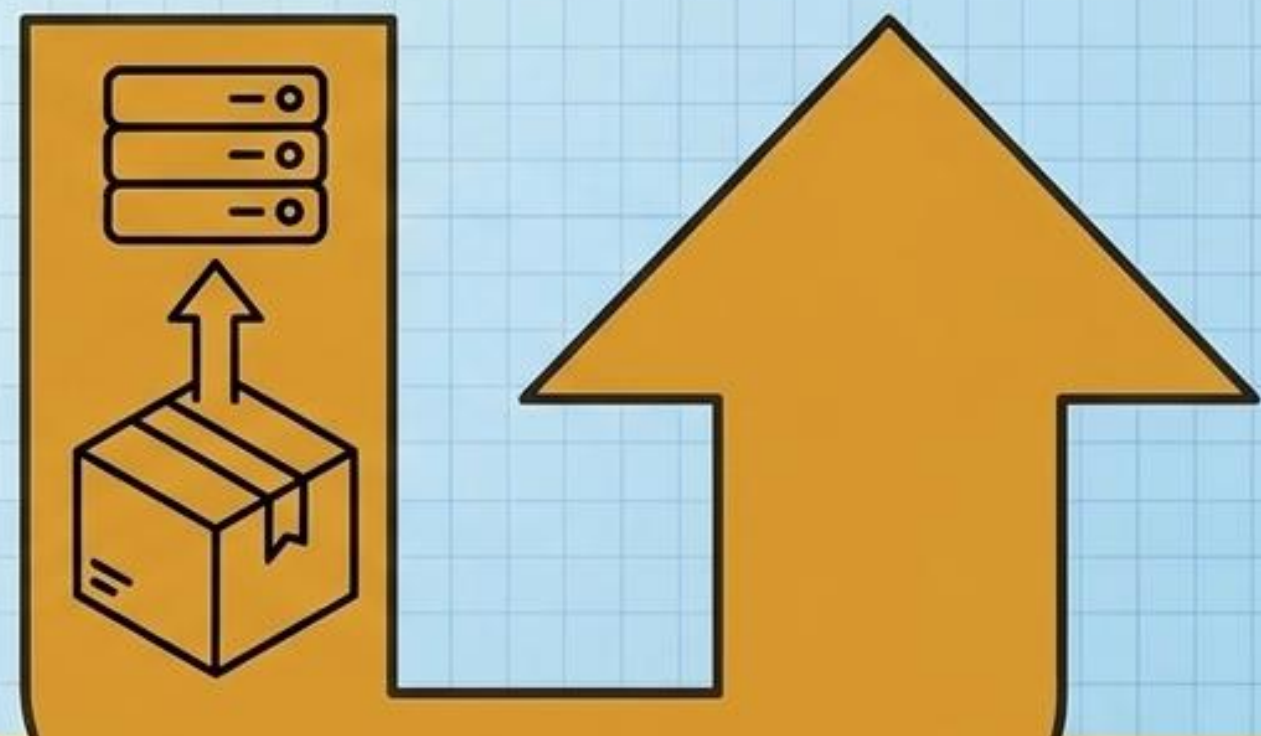
Pílula de Aprendizado: O Método POST

Até agora, atuamos apenas como leitores da web (método GET automático). Mas como fazemos para preencher um formulário, criar um usuário ou salvar uma mensagem no banco de dados da nuvem?

A Resposta: Precisamos empacotar nossos dados e enviá-los usando uma requisição **POST**.



GET (Receber/Ler)

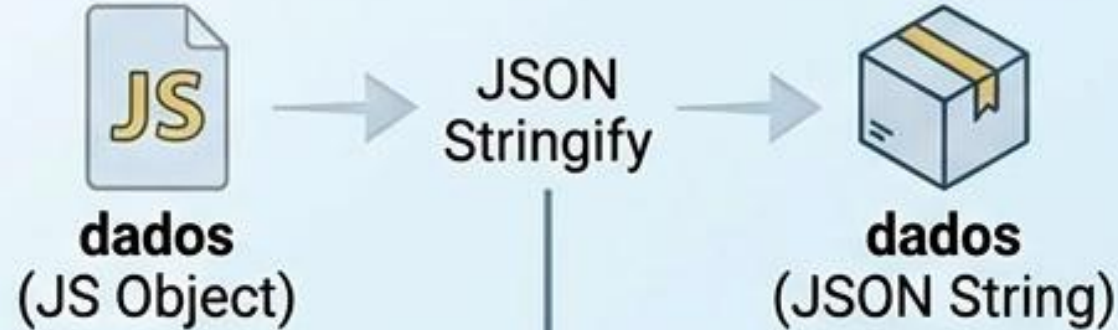


POST (Enviar/Escriver)

A Sintaxe do POST

```
fetch('https://api.exemplo.com/dados', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify(dados)  
});
```

O Método:
Mudando a
intenção de
"quero ler" para
"quero enviar".



O Cabeçalho:
Avisando ao
servidor que o
idioma da
nossa
encomenda é o
formato JSON.

O Corpo: Convertendo nosso objeto JavaScript
em um pacote de texto que pode viajar pela rede.

Atividade 3 em Duplas: CRUD de Contatos

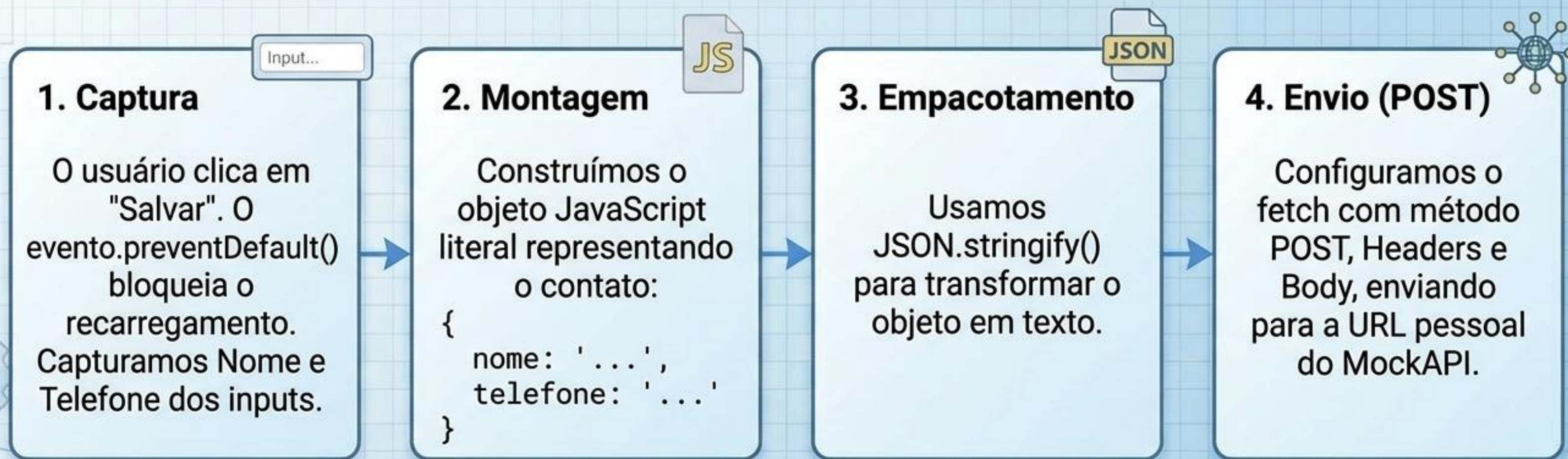
Vamos construir a fundação de um sistema real: Criação e Leitura de dados na nuvem usando a plataforma MockAPI.io.

Foco total no 'C' (Create/POST)
e no 'R' (Read/GET)



Atividade 3: Enviando um Novo Contato

O Fluxo de Ação:



Requisito Bônus: A Mágica do Auto-Refresh

O Desafio: Após o POST ser concluído com sucesso, a **lista de contatos na tela deve exibir o novo contato imediatamente**, sem que o **usuário precise apertar F5**.

A Solução Lógica: Dentro do `.then()` (ou após o `await`) da função de criação, basta invocar a função que busca e renderiza a lista de contatos!



Metodologia de Entrega Oficial

Para validar a presença e avaliação, entregue **exatamente 9 arquivos** anexados na tarefa do Google Sala de Aula.

Pasta 1: Validador_CEP

- index.html
- style.css
- script.js

Pasta 2: Notícias_IBGE

- index.html
- style.css
- script.js

Pasta 3: CRUD_Contatos

- index.html
- style.css
- script.js

Resumo Massivo de Comandos (Guia Definitivo)

DOM e UI

```
document.querySelector()  
element.innerHTML  
element.classList.add()
```

Eventos

```
addEventListener('click')  
evento.preventDefault()
```

Arrays e Lógica

```
array.map()  
  
try { ... } catch (erro)  
{ ... }
```

Rede (APIs)

GET:

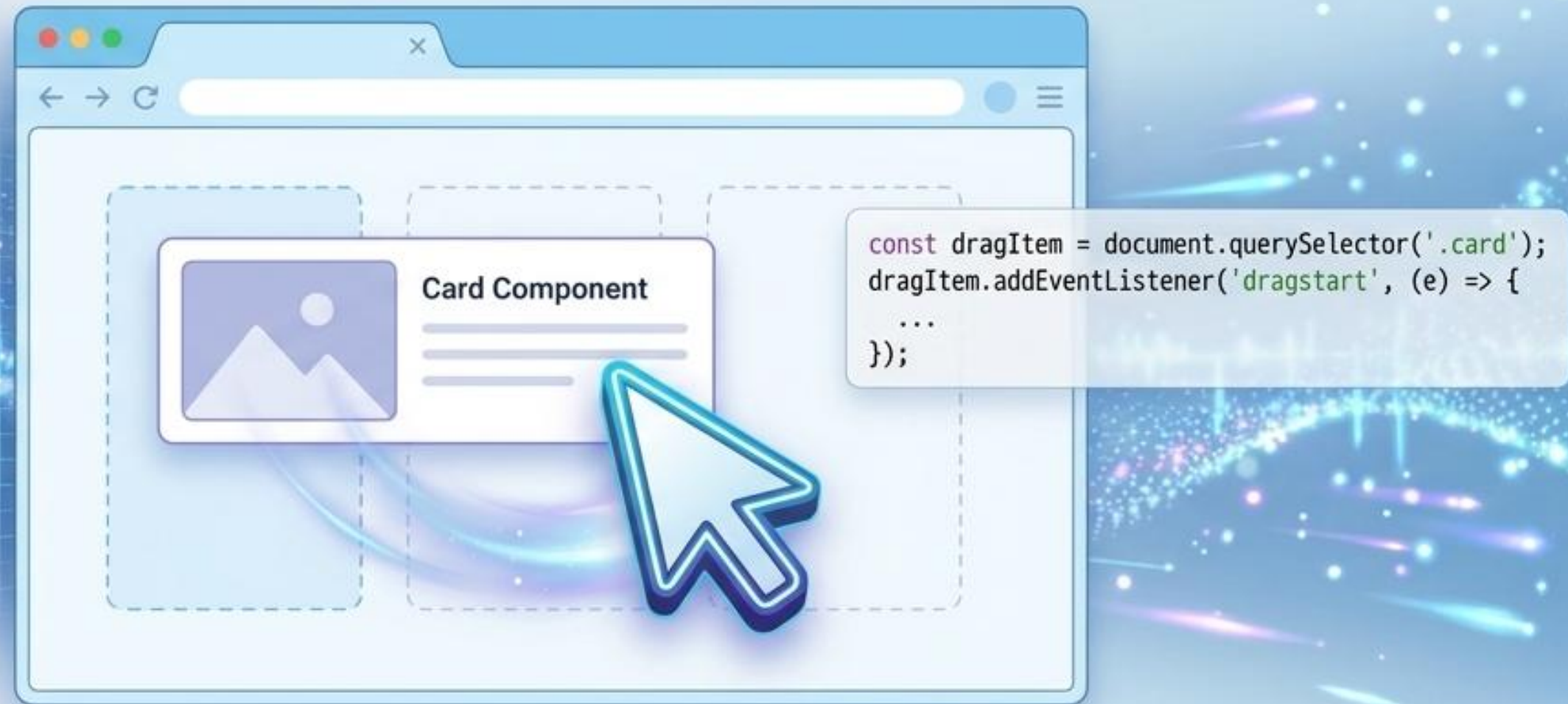
```
const resp = await  
fetch(url);  
const dados = await  
resp.json();
```

POST:

```
fetch(url, {  
  method: 'POST',  
  body:  
    JSON.stringify(obj)  
})
```


Missão Cumprida: Você se conectou ao mundo.

O que vem a seguir? Dando movimento à interface...



Hoje, nossas aplicações aprenderam a buscar informações em qualquer servidor do planeta. Na próxima aula, vamos explorar o poder interno do navegador: **APIs Nativas**. Prepare-se para dominar a **Drag and Drop API** e capturar imagens da sua webcam!